

Retry Policy Gradients in Continuous Action Spaces

Anonymous authors
Paper under double-blind review

Keywords: RLJ, RLC, formatting guide, style file, L^AT_EX template.

Summary

Retry-based objectives such as `pass@k` and `max@k` optimize the maximum return over multiple trajectories, and recent work has shown that they can promote exploration without explicit exploration bonuses. In discrete action spaces, ReMax was shown to do so by adapting to return uncertainty. In this work, we introduce pathwise derivative estimators for retry objectives and use them to extend ReMax to continuous action spaces. We study the resulting learning dynamics and show that ReMax can encourage exploration even with deterministic rewards by reshaping the policy-gradient landscape. In particular, it alters gradients both in direction, biasing updates toward higher policy entropy, and in magnitude, damping gradients and slowing convergence. We further show that Adam’s normalization mitigates this damping. Empirically, we instantiate **ReMax Actor-Critic (ReMAC)**, an off-policy actor-critic algorithm that optimizes the ReMax objective using a pathwise derivative estimator. Our experiments provide empirical evidence that these gradient-level insights also appear in practical deep RL training.

Contribution(s)

1. We extend the retry-based objective, ReMax, to continuous action spaces and instantiate it in an off-policy actor-critic framework as ReMax Actor-Critic (ReMAC) using a pathwise derivative estimator.

Context: ReMax was originally proposed for discrete action spaces (Koyamada et al., 2023; Nishimori et al., 2026), where the policy gradient can be computed exactly. A parallel line of work, `pass@K` optimization (Walder & Karkhanis, 2025; Tang et al., 2025; Chen et al., 2025) for LLM post-training, relies on sample-based gradient estimators because the candidate space is extremely large. However, since LLM post-training typically involves non-differentiable reward functions, pathwise gradient estimators have not been explored.

2. We analyze the optimization dynamics induced by the ReMax objective through toy examples and theoretical analysis. The analysis reveals ReMax reshapes the gradient landscape in both direction and magnitude: it can encourage higher policy entropy when the policy is far from the optimum and has low entropy, and damp gradient norms near the optimum.

Context: Previous work showed that retry-based objectives can promote exploration in response to return uncertainty (Koyamada et al., 2023; Nishimori et al., 2026). Our analysis instead focuses on how retry objectives shape policy gradients. While gradient damping was briefly noted by Nishimori et al. (2026), we provide a theoretical explanation and discuss how Adam’s normalization interacts with it, in addition to the directional property.

3. We provide empirical evidence that ReMAC integrates into modern continuous-control pipelines and achieves performance comparable to Soft Actor-Critic (SAC) (Haarnoja et al., 2018), while exhibiting the entropy-related behaviors predicted by our analysis.

Context: Our experiments serve as proof-of-concept validation rather than a comprehensive exploration benchmark. As the first application of ReMax to continuous control, they verify the practical feasibility of the objective and examine whether the predicted behaviors of the entropy appear in deep RL training.

Retry Policy Gradients in Continuous Action Spaces

Anonymous authors

Paper under double-blind review

Abstract

1 Retry-based objectives such as $\text{pass}@k$ and $\text{max}@k$ optimize the maximum return over
 2 multiple trajectories, and recent work has shown that they can promote exploration
 3 without explicit exploration bonuses. In discrete action spaces, ReMax was shown to
 4 do so by adapting to return uncertainty. In this work, we introduce pathwise deriva-
 5 tive estimators for retry objectives and use them to extend ReMax to continuous action
 6 spaces. We study the resulting learning dynamics and show that ReMax can encourage
 7 exploration even with deterministic rewards by reshaping the policy-gradient landscape.
 8 In particular, it alters gradients both in direction, biasing updates toward higher policy
 9 entropy, and in magnitude, damping gradients and slowing convergence. We further
 10 show that Adam’s normalization mitigates this damping. Empirically, we instantiate
 11 **ReMax Actor-Critic (ReMAC)**, an off-policy actor-critic algorithm that optimizes the
 12 ReMax objective using a pathwise derivative estimator. Our experiments provide empir-
 13 ical evidence that these gradient-level insights also appear in practical deep RL training.

1 Introduction

15 Exploration remains one of the central challenges in reinforcement learning (RL) (Sutton & Barto,
 16 2018). In continuous-control tasks, exploration is typically induced through stochastic poli-
 17 cies together with entropy regularization, as in Soft Actor-Critic (SAC) (Haarnoja et al., 2018;
 18 Christodoulou, 2019). Noise injection mechanisms such as action noise or parameter-space noise
 19 are also commonly used (Fujimoto et al., 2018; Lillicrap et al., 2020; Plappert et al., 2017). Recent
 20 work has further improved actor-critic methods through architectural refinements such as ensemble
 21 critics (Chen et al., 2021; Hiraoka et al., 2021).

22 Recently, retry-based objectives—which maximize the best return over multiple ($M \in \mathbb{N}$) trajec-
 23 tories—have been proposed as an alternative formulation for RL in discrete action spaces (Koyamada
 24 et al., 2023; Nishimori et al., 2026; Tang et al., 2025; Walder & Karkhanis, 2025). These objectives
 25 are motivated by the observation that when an agent can attempt the same task multiple times, ex-
 26 ploration may arise naturally from maximizing the best outcome among repeated trials. In episodic
 27 RL settings, prior work introduced the ReMax objective and showed empirically that it can promote
 28 exploration in discrete action spaces (Koyamada et al., 2023; Nishimori et al., 2026).

29 Despite these promising results, retry-based objectives have so far been studied primarily in dis-
 30 crete action spaces. Extending these ideas to continuous control presents additional challenges,
 31 particularly in computing policy gradients when the action space is continuous. Furthermore, the
 32 mechanisms by which retry objectives influence exploration behavior are not fully understood.

33 Prior work focused explicitly on (epistemic) uncertainty in returns and showed that ReMax can make
 34 the optimal policy exploratory in response to such uncertainty (Koyamada et al., 2023; Nishimori
 35 et al., 2026). In this work, we identify inherent exploration-promoting properties in both the *di-*
 36 *rection* and *magnitude* of the ReMax gradient when $M > 1$. Specifically, when the policy is far
 37 from the optimum and has low entropy, the gradient can point toward increasing the policy entropy.
 38 Moreover, as the retry budget increases, the gradient norm near the optimum decreases, which slows
 39 convergence and can further encourage exploration.

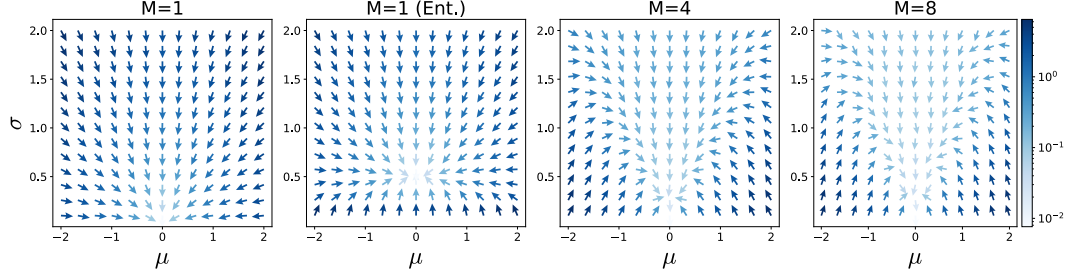


Figure 1: Vector field of normalized ReMax gradients for $M = 1, 4, 8$ over $\mu \in [-2, 2]$ and $\sigma \in [0.1, 2]$. We also plot the entropy-regularized standard RL ($M = 1$ (Ent.)). Arrow color indicates gradient norm (darker is larger). For larger M , the gradient increases σ when μ is far from the optimum (bottom edge) and the gradient norm near the optimum shrinks, slowing convergence. Unlike entropy regularization, ReMax always converges to the deterministic optimum $\mu = 0, \sigma = 0$.

Fig. 1 illustrates this effect. It shows the vector field of ReMax gradients for a 1D Gaussian policy $a \sim \mathcal{N}(\mu, \sigma)$ with reward $r(a) = -a^2$. For larger M , when μ is far from the optimum and σ is small (bottom edges), the gradient points toward increasing σ , whereas $M = 1$ (standard RL) always decreases σ . Conversely, when μ is near the optimum and σ is large (top center), the gradient norm is damped (lighter color), slowing convergence. Unlike entropy regularization, ReMax does not change the optimal parameters $\mu = 0$ and $\sigma = 0$, whereas entropy regularization shifts the optimum to $\mu = 0$ and $\sigma > 0$. These properties yield an optimization trajectory in which policy entropy first increases and then gradually decreases, encouraging exploration before converging to the deterministic optimal policy. A detailed explanation of Fig. 1 is provided in Sec. 5.1, and a theoretical explanation for why these properties emerge is given in Sec. 5.2. Finally, the damping effect can be mitigated by Adam’s adaptive gradient normalization (Kingma & Ba, 2014), the de facto standard optimizer in deep learning.

Finally, we instantiate the ReMax objective in an off-policy actor–critic algorithm, which we call **ReMax Actor-Critic (ReMAC)**. Empirically, we show that ReMAC achieves performance comparable to SAC while exhibiting the predicted entropy behavior. Importantly, our goal is not to propose a new state-of-the-art exploration algorithm for continuous control. Instead, we aim to understand how retry-based objectives influence the optimization dynamics of policy-gradient methods and to demonstrate their practical viability in deep RL pipelines. Because ReMax modifies the objective rather than the policy parameterization or reward function, it is also potentially complementary to existing exploration methods such as noise-based exploration or ensemble-based critics.

2 Related Work

A large body of work studies exploration strategies in RL. One common approach augments the reward with intrinsic bonuses that encourage agents to visit less-explored states. Examples include count-based exploration (Bellemare et al., 2016; Ostrovski et al., 2017; Lobel et al., 2023; Tang et al., 2017) and prediction-error-based bonuses (Burda et al., 2019; Pathak et al., 2017). Other approaches use information gain or curiosity signals as intrinsic rewards (Houthoofd et al., 2016; Sukhija et al., 2024; Schmidhuber, 2010). Another line of work promotes exploration by modeling uncertainty in value estimates. Ensemble-based methods approximate posterior sampling using multiple Q-functions (Osband et al., 2016; 2018; 2019; 2023; Zhang & Yao, 2019), while related approaches use Langevin Monte Carlo updates (Ishfaq et al., 2024; 2025).

In continuous-control settings, exploration is typically induced through stochastic policies with entropy regularization, as in SAC (Haarnoja et al., 2018; Christodoulou, 2019; Lee et al., 2024; Naudman et al., 2024). Exploration can also be encouraged through noise injection mechanisms such as action noise or parameter-space noise (Fujimoto et al., 2018; Lillicrap et al., 2020; Plappert et al.,

2017; Fortunato et al., 2017). Recent work has further improved actor-critic methods through architectural refinements, including ensemble-based critics such as Randomized Ensembled Double Q-learning (REDQ) (Chen et al., 2021) and Dropout Q-functions (DroQ) (Hiraoka et al., 2021).

Retry-based objectives provide a different perspective on exploration. Instead of modifying the reward or injecting noise into the policy, these objectives evaluate the best outcome among multiple sampled trajectories. Prior work showed that retry objectives can induce exploratory behavior in discrete action spaces (Koyamada et al., 2023; Nishimori et al., 2026; Walder & Karkhanis, 2025; Chen et al., 2025; Tang et al., 2025; Hamid et al., 2025). Koyamada et al. (2023) proposed a policy-gradient estimator for ReMax using a resettable simulator. Nishimori et al. (2026) developed a formulation based on Q-functions that enables analytical gradient computation in relatively small discrete action spaces. Related ideas have also appeared in the literature on LLM post-training. Several works optimize pass@K or its generalization max@K to improve output diversity (Tang et al., 2025; Chen et al., 2025; Walder & Karkhanis, 2025). The max@K objective (Walder & Karkhanis, 2025) can be viewed as a special case of ReMax when rewards are deterministic.

In this study, we build on these works by extending ReMax to continuous control. Compared to Nishimori et al. (2026); Koyamada et al. (2023), we not only extend ReMax to continuous action spaces but also analyze how it promotes exploratory behavior even with deterministic rewards by reshaping the gradient landscape. Although Nishimori et al. (2026) noted gradient damping under the ReMax objective, we reveal an additional directional property of the gradient that can encourage higher policy entropy and provide a theoretical explanation for why these properties emerge in continuous-control settings. Relative to the pass@K line of work, we consider episodic returns that require function approximation to emulate retries, whereas pass@K assumes explicit retries for a prompt and directly obtains rewards. While pass@K-based works also develop sample-based REINFORCE-style gradient estimators, we adopt a pathwise gradient estimator (Kingma & Welling, 2013; Parmas & Sugiyama, 2021; Jankowiak & Obermeyer, 2018; Rezende et al., 2014). This estimator is not applicable in typical LLM post-training settings, where rewards are produced by non-differentiable rule-based verifiers. Unlike bonus-based exploration methods such as entropy regularization, retry-based objectives modify the optimization objective by evaluating the best outcome among multiple sampled actions without biasing the optimal policy under the standard RL objective. This orthogonality can potentially provide an opportunity to combine ReMax with existing exploration methods. Moreover, preserving the optimal policy under the standard RL objective removes the need to carefully tune bonus coefficients to mitigate bias.

3 Preliminaries

Reinforcement learning (RL). We consider the reinforcement learning (RL) problem, formulated as the Markov Decision Process (MDP) $\mathcal{M} = (\mathcal{S}, \mathcal{A}, \mathcal{P}, p_0, \mathcal{R}, \gamma)$, where $\mathcal{S}$ is the state space, $\mathcal{A} \subseteq \mathbb{R}^d$ (d is the dimension of the action space) is the continuous action space, $\mathcal{P}$ is the transition dynamics, p_0 is the initial state distribution, $\mathcal{R} : \mathcal{S} \times \mathcal{A} \rightarrow \mathbb{R}$ is the reward function, and $\gamma \in [0, 1)$ is the discount factor. The goal is to train a policy $\pi(\cdot | s)$ that maximizes the expected discounted reward $\mathbb{E}_\pi[R_0]$, where $R_0 = \sum_{h=0}^{\infty} \gamma^h r(s_h, a_h)$ is the discounted return and $\mathbb{E}_\pi[\cdot]$ denotes the expectation over the trajectory $(s_0, a_0, r_0, s_1, a_1, r_1, \dots)$ under policy π . We define the value function of the policy π as $V^\pi(s) = \mathbb{E}_\pi[R_0 | s_0 = s]$ and the Q-function as $Q^\pi(s, a) = \mathbb{E}_\pi[R_0 | s_0 = s, a_0 = a]$.

4 ReMax Objective and its Gradient Estimation

This section introduces the ReMax objective and describes how we estimate its policy gradient.

4.1 ReMax Objective

Recent work has shown that optimizing the return over multiple trajectories can promote exploration without explicit exploration bonuses in RL (Walder & Karkhanis, 2025; Nishimori et al., 2026; Koyamada et al., 2023). In episodic RL, directly optimizing such a multi-trajectory return typically assumes the ability to reset the environment at arbitrary time steps (Koyamada et al., 2023), which is often infeasible in continuous-control settings. We therefore adopt the Q-function-based (deterministic) ReMax formulation (Nishimori et al., 2026):

$$\mathcal{J}^M(\pi) := \mathbb{E}_{a_{1:M} \sim \pi, s \sim p_0} \left[\max_{m=1:M} Q^\pi(s, a_m) \right], \quad (1)$$

where $M \in \mathbb{N}$ is the number of retries controlling the exploration level and π is the policy. When $M = 1$, the objective reduces to the standard RL objective.

Importantly, the ReMax objective does not introduce bias with respect to the standard RL objective because it preserves the optimal policy. Since the optimal policy of an MDP is deterministic, let $\pi^* \in \arg \max_\pi J^1(\pi)$ denote an optimal deterministic policy. Then $\pi^* \in \arg \max_\pi \mathcal{J}^M(\pi)$ for any $1 \leq M < \infty$. This follows because for any state s and policy π , $\mathbb{E}_{a_{1:M} \sim \pi} [\max_{m=1:M} Q^\pi(s, a_m)] \leq \max_a Q^\pi(s, a) = \mathbb{E}_{a_{1:M} \sim \pi^*} [\max_{m=1:M} Q^\pi(s, a_m)]$. This property distinguishes ReMax from entropy-regularized objectives, which shift the optimal policy toward higher-entropy distributions to balance exploration and exploitation and thus introduce bias in the final solution.

4.2 Gradient Estimation of the ReMax Objective

We optimize Eq. (1) using policy-gradient methods. Nishimori et al. (2026) showed how to compute Eq. (1) and its gradient exactly in discrete action spaces using the policy vector and Q-values. However, this approach does not extend to continuous action spaces because it requires sorting Q-values over all actions. We therefore use sample-based gradient estimators (Parmas et al., 2018; Parmas, 2018). Let π_θ denote a policy parameterized by $\theta \in \mathbb{R}^p$. For a given state s , we sample an action batch $a_{1:B} := (a_1, \dots, a_B) \sim \pi_\theta(\cdot | s)$ with $B \geq M$ and compute the corresponding Q-values $q_{1:B} := (q_1, \dots, q_B)$, where $q_i = Q^{\pi_\theta}(s, a_i)$. In practice, Q^{π_θ} is represented by a function approximator such as a neural network. We consider two approaches for gradient estimation.

Likelihood ratio (LR) gradient estimator. The likelihood-ratio estimator approximates the gradient of $\mathbb{E}_{a \sim \pi_\theta} [f(a)]$ ($f : \mathcal{A} \rightarrow \mathbb{R}$) as $\mathbb{E}_{a \sim \pi_\theta} [\nabla_\theta \log \pi_\theta(a | s) f(a)]$, also known as the REINFORCE estimator (Williams, 1992). Because the M actions are coupled through $\max_{m=1:M}$, it is not directly applicable to Eq. (1). Nevertheless, Walder & Karkhanis (2025) derived an LR-style estimator for deterministic ReMax: $\hat{g} = \frac{1}{B} \sum_{i=1}^B \nabla_\theta \log \pi_\theta(a_i | s) \hat{A}_i$, where $\hat{A}_i$ denotes an advantage estimate. We refer the reader to Walder & Karkhanis (2025) for details.

Reparameterization (RP) gradient estimator. Alternatively, we can estimate the gradient using the reparameterization trick (Kingma & Welling, 2013). Assume π_θ is Gaussian with mean μ and standard deviation σ . Actions are sampled as $a_i(\mu, \sigma) = \mu + \sigma \varepsilon_i$, where $\varepsilon_i \sim \mathcal{N}(0, 1)$. Thus $\frac{\partial a_i}{\partial \mu} = 1$ and $\frac{\partial a_i}{\partial \sigma} = \varepsilon_i$. For higher-dimensional actions, see Rezende et al. (2014). When Q^{π_θ} is differentiable, we can compute $\nabla_{a_i} Q^{\pi_\theta}(s, a_i)$ and backpropagate through q_i . Walder & Karkhanis (2025) proposed an unbiased estimator of Eq. (1) from $q_{1:B}$ (sorted in ascending order):

$$\rho^M(q_{1:B}) := \frac{1}{\binom{B}{M}} \sum_{i=1}^B w_i q_i, \quad (2)$$

where $w_i = \binom{i-1}{M-1}$ is the probability of the i -th action being the best among M actions. Because ρ^M is linear in $q_{1:B}$, $\nabla_\theta \rho^M(q_{1:B})$ can be obtained via automatic differentiation. Walder & Karkhanis

(2025) did not explore this RP estimator because their focus was LLM post-training, where rewards are typically non-differentiable.

Our choice. Both estimators have different trade-offs. LR gradients do not require a differentiable Q-function and can remain unbiased with raw returns, which suits on-policy learning, but they often suffer from high variance (Greensmith et al., 2004). RP gradients require a differentiable Q-function but typically yield lower-variance estimates (Parmas et al., 2018). In continuous control, off-policy actor-critic methods (Degris et al., 2012; Haarnoja et al., 2018; Fujimoto et al., 2018) are widely used for their sample efficiency. These methods rely on replay buffers storing transitions from past behavior policies and update both the Q-function and policy in an off-policy manner. Consequently, obtaining raw-return targets for arbitrary state-action pairs is difficult, weakening the main advantage of LR estimators. Thus LR offers limited benefits in this setting while retaining high variance, whereas RP estimators are standard and effective. We therefore adopt the RP estimator.

5 Gradient Analysis

We analyze how the ReMax objective promotes exploration through policy gradients. As illustrated by Nishimori et al. (2026), ReMax can induce exploratory behavior by adapting to reward stochasticity or nonstationarity. Here we show that ReMax can promote exploration even with *deterministic* rewards by reshaping the gradient landscape so that the policy maintains higher entropy along the optimization path. When the policy is far from the optimum and has low entropy, the gradient encourages higher entropy (direction), while near the optimum the gradient norm is damped (magnitude). We then provide a theoretical explanation for these properties (Sec. 5.2) and show that Adam’s normalization can mitigate the damping effect (Sec. 5.3), providing practical guidance for optimizing ReMax in continuous-control settings.

To illustrate this phenomenon, we consider a toy example with a one-dimensional Gaussian policy parameterized by mean $\mu \in \mathbb{R}$ and standard deviation $\sigma > 0$, i.e., $a \sim \mathcal{N}(\mu, \sigma)$. We use the quadratic reward $r(a) = -a^2$.

5.1 Vector field analysis

In Fig. 1, we visualize the vector field of the RP gradient estimator (Sec. 4.2) with batch size $B = 16$ over $\mu \in [-2, 2]$ and $\sigma \in [0.1, 2]$, varying $M \in \{1, 4, 8\}$. For reference, we also plot the entropy-regularized objective for $M = 1$ (standard RL with entropy regularization) $r(a) = -a^2 + \alpha \frac{1}{2} \log(2\pi\sigma^2)$ with $\alpha = 0.5$. For $M = 1$, the gradient with respect to σ is always negative. For $M > 1$, the ReMax objective exhibits two exploration-promoting properties:

- **Entropy Increase (direction):** When the mean is far from the optimum and σ is small, the gradient increases σ , inducing more exploratory behavior.
- **Gradient Damping (magnitude):** As M increases, the gradient norm near the optimum (μ small and σ large) decreases, slowing convergence and promoting exploration.

More details are provided in App. B.1. Furthermore, ReMax converges to the deterministic optimal policy $(\mu, \sigma) = (0, 0)$, whereas entropy regularization converges to $(0, 0.5)$. Together these properties keep entropy high during optimization: entropy first increases due to the directional effect and then decreases gradually due to gradient damping.

5.2 Theoretical Explanation

We now provide intuition for how ReMax with $M > 1$ shapes exploration dynamics. As observed in Sec. 5.1, the gradient with respect to σ depends on the distance between the policy mean μ and the optimum. When the policy is far from the optimum and σ is small, larger σ and multiple samples increase the probability of generating a “lucky” action closer to the optimum, producing a positive gradient that increases entropy. Near the optimum, taking the minimum over M samples ensures

that at least one action lies close to the optimum. This flattens the local loss landscape and damps the gradient, preventing entropy from collapsing prematurely.

Setup and assumptions. Let $A = \mu + \sigma\varepsilon$ with $\varepsilon \sim \mathcal{N}(0, 1)$. We consider returns of the form $r(a) = -c(a)$ with $c : \mathbb{R} \rightarrow \mathbb{R}$. For $M \geq 1$, define $J^M(\mu, \sigma) = \mathbb{E}[\max_{m=1:M} -c(A_m)]$ with $A_m = \mu + \sigma\varepsilon_m$ and $\varepsilon_m \stackrel{i.i.d.}{\sim} \mathcal{N}(0, 1)$. Let $m^* \in \arg \min_{m=1:M} c(A_m)$ denote the selected action. The goal of the following analysis is to nurture intuition for how the ReMax objective shapes optimization dynamics. We consider several assumptions on the cost function c . These assumptions are not intended to model deep RL environments globally; rather, they are introduced to make the analysis tractable. They allow us to study how the ReMax objective affects the gradient landscape in a simplified setting.

Assumption 1 (Differentiability and optima). c is twice differentiable, $c(0) = 0$, and $c'(0) = 0$.

Assumption 2 (Strong Convexity). c is λ -strongly convex, i.e., $c''(a) \geq \lambda > 0$.

Assumption 3 (Smoothness). c is L -smooth, i.e., $|c'(a) - c'(b)| \leq L|a - b|$.

Entropy increase effect for $M > 1$. We show that for $M > 1$, the ReMax objective can increase policy entropy when the mean is far from the optimum.

Proposition 1 (Entropy Increase Effect). Assume Assumptions 1, 2, and 3 hold. Let $\varepsilon_{(1)} = \min_{m=1:M} \varepsilon_m$ and define the event $E = \{\mu + \sigma\varepsilon_{(1)} \geq 0\}$. Then, the gradient of the ReMax objective with respect to $\sigma > 0$ satisfies:

$$\partial_\sigma J^M(\mu, \sigma) \geq \mu \mathbb{E}[(\lambda \varepsilon_{(1)}^- - L \varepsilon_{(1)}^+) \mathbb{I}_E] - \sigma \max(\lambda, L) \mathbb{E}[\varepsilon_{(1)}^2 \mathbb{I}_E] + \mathbb{E}[-c'(A_{m^*}) \varepsilon_{m^*} \mathbb{I}_{E^c}],$$

where $\varepsilon_{(1)}^- = \max(0, -\varepsilon_{(1)})$ and $\varepsilon_{(1)}^+ = \max(0, \varepsilon_{(1)})$.

The proof is given in App. A.2. Assume that μ is far from the optimum (e.g., $\mu \gg 0$) and σ is small so that $\mathbb{P}(E) \approx 1$. We therefore focus on the first two terms in the bound. When sampling $M > 1$ actions, the minimum noise $\varepsilon_{(1)}$ is likely to be negative, making the term $\mu \mathbb{E}[\lambda \varepsilon_{(1)}^-]$ positive and thus $\partial_\sigma J^M(\mu, \sigma)$ positive. This reflects the mechanism whereby *increasing* σ moves the selected action $A_{m^*} = \mu + \sigma\varepsilon_{(1)}$ closer to the optimum 0. As M increases, the favorable linear term typically becomes stronger, although the quadratic penalty and the contribution from E^c may also grow. Hence, a stronger entropy-increasing tendency is expected in regimes where $|\mu|/\sigma$ is large and σ is sufficiently small. As shown in App. A.3, the same argument holds when $\mu \ll 0$. Furthermore, the proposition below shows that the gradient with respect to σ is strictly negative when $M = 1$.

Proposition 2 (Entropy Decrease for $M = 1$). Assume Assumptions 1, 2, and 3 hold. For $M = 1$, $\mu \in \mathbb{R}$ and $\sigma > 0$, the gradient of the objective $J^1(\mu, \sigma) = -\mathbb{E}[c(\mu + \sigma\varepsilon)]$ with respect to σ satisfies

$$\partial_\sigma J^1(\mu, \sigma) \leq -\lambda\sigma < 0.$$

The proof is given in App. A.1. This result contrasts the behavior of standard RL ($M = 1$) and ReMax ($M > 1$).

Gradient damping effect for $M > 1$. We next bound the gradient magnitudes by the expected minimum absolute deviation.

Proposition 3 (Gradient Damping). Assume Assumptions 1, 2, and 3 hold. Let $A_m = \mu + \sigma\varepsilon_m$. Then, the magnitudes of the gradients with respect to μ and σ are bounded by

$$|\partial_\mu J^M(\mu, \sigma)| \leq L \sqrt{\frac{L}{\lambda}} \mathbb{E}\left[\min_{m=1:M} |A_m|\right], \quad |\partial_\sigma J^M(\mu, \sigma)| \leq L \sqrt{\frac{L}{\lambda}} \mathbb{E}\left[\min_{m=1:M} |A_m| |\varepsilon_{m^*}|\right].$$

The proof is given in App. A.4. These bounds capture the "forgiveness" of the minimum operator depending on the distance to the optimum. In standard RL ($M = 1$), maintaining a large σ near the

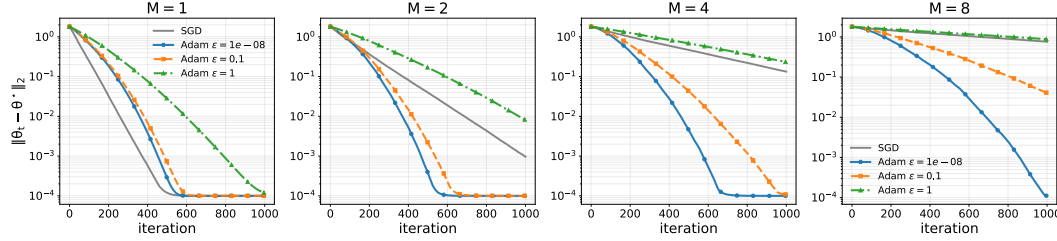


Figure 2: The distance to the optimal parameter $(\mu, \sigma) = (0, 0)$ for Adam with different ε values and for SGD. As the vector field indicates, the convergence speed slows as M increases in every case. However, with smaller ε the convergence near the optimum remains fast, whereas larger ε aligns the trajectory with the SGD path that follows the vector field.

234 optimum often produces bad actions, generating strong negative gradients that rapidly shrink σ . In
 235 contrast, ReMax evaluates only the best action. When μ is near 0, increasing M makes it more likely
 236 that at least one sampled action falls close to the optimum. Thus, for fixed σ (or when M is large
 237 relative to σ), the minimum deviation can be small, which damps the gradients. Conversely, when
 238 $|\mu| \gg \sigma$, the minimum deviation $\min_{m=1:M} |A_m| \approx |\mu|$ remains large, alleviating the damping
 239 effect and allowing the rapid updates described in Proposition 1.

240 5.3 Effect of the normalization by Adam

241 Although the vector-field analysis suggests that ReMax with $M > 1$ induces a more exploration-
 242 encouraging gradient field than standard RL, the optimization path does not necessarily follow it.
 243 In particular, with Adam (Kingma & Ba, 2014), normalization can counteract the damping effect,
 244 leading to faster convergence with decreasing entropy. Adam updates parameters as $\theta_{t+1} = \theta_t -$
 245 $\alpha \frac{\hat{m}_t}{\sqrt{\hat{v}_t + \varepsilon}}$, where $\hat{m}_t$ and $\hat{v}_t$ are moving averages of the gradient and squared gradient and $\varepsilon > 0$ is a
 246 small constant to prevent division by zero. If ε is negligible, then $\frac{\hat{m}_t}{\sqrt{\hat{v}_t + \varepsilon}} \approx \frac{\hat{m}_t}{\sqrt{\hat{v}_t}}$, which effectively
 247 normalizes the gradient and reduces the damping effect.

248 To observe this behavior, we optimize the policy $a \sim \mathcal{N}(\mu, \sigma^2)$ for the reward function $r(a) = -a^2$
 249 with Adam using $\varepsilon \in \{1e-8, 1e-1, 1.0\}$. We set $\alpha = 0.01$, $B = 64$, $T = 1000$, and $M \in$
 250 $\{1, 2, 4, 8\}$, starting from $(\mu, \sigma) = (-1.5, 1.0)$. Fig. 2 shows the distance to the optimal parameter
 251 $(0, 0)$ for Adam and SGD. As suggested by the vector field, convergence slows as M increases.
 252 However, smaller ε mitigates the damping effect: with $M = 4, 8$, convergence with $\varepsilon = 1e-8$ is
 253 much faster than SGD, whereas $\varepsilon = 1.0$ slows convergence to match SGD, which follows the vector
 254 field. These results suggest a practical recipe for optimizing ReMax with Adam. Adam naturally
 255 benefits from the *directional* entropy-increase effect, but with the default $\varepsilon = 1e-8$ the damping
 256 effect is muted. Increasing ε can recover it, though larger ε also reduces the effective step size and
 257 slows convergence. Thus jointly tuning ε and the learning rate may provide better control of policy
 258 entropy during training. More details are provided in App. B.2.

259 6 ReMax Actor-Critic Algorithm

260 Having observed an exploration-encouraging property of ReMax, we instantiate it in a simple off-
 261 policy actor-critic algorithm with a critic Q_ϕ and a stochastic policy π_θ . The critic is trained with
 262 temporal-difference regression on transitions sampled from a replay buffer $\mathcal{D}$, while the actor is
 263 optimized using the ReMax computed from batches of action samples.

264 **Critic update.** Given a transition $(s, a, r, s') \sim \mathcal{D}$, we define the target as $y = r + \gamma Q_\phi(s', a')$,
 265 where $a' \sim \pi_\theta(\cdot | s')$ and Q_ϕ is a target critic. The critic is updated by minimizing the squared
 266 Bellman error:

$$L_Q(\phi) := \mathbb{E}[(Q_\phi(s, a) - y)^2]. \quad (3)$$

Algorithm 1 ReMAC

Require: Retry parameter M , action batch sizes B , discount γ , learning rate α, β , target critic update rate τ .

- 1: Initialize policy π_θ , critics Q_{ϕ_1}, Q_{ϕ_2} , target critics $Q_{\bar{\phi}_1}, Q_{\bar{\phi}_2}$, and replay buffer $\mathcal{D}$
- 2: **for** each iteration t **do**
- 3: **for** each environment step **do**
- 4: $a \sim \pi_\theta(\cdot|s), (r, s') \sim \mathcal{P}(s, a)$ ▷ interact with environment
- 5: $\mathcal{D} \leftarrow \mathcal{D} \cup \{(s, a, r, s')\}$ ▷ store transition
- 6: **end for**
- 7: **for** each gradient step **do**
- 8: $(s, a, r, s') \sim \mathcal{D}, a' \sim \pi_\theta(\cdot|s')$ ▷ sample minibatch and next action
- 9: $y \leftarrow r + \gamma \min(Q_{\bar{\phi}_1}(s', a'), Q_{\bar{\phi}_2}(s', a'))$ ▷ target value
- 10: $\phi_j \leftarrow \phi_j - \alpha \nabla_{\phi_j} (Q_{\phi_j}(s, a) - y)^2, j \in \{1, 2\}$ ▷ update critics
- 11: $a_{1:B} \sim \pi_\theta(\cdot|s), q_i \leftarrow \min_{j=1,2} Q_{\phi_j}(s, a_i) \forall i \in \{1, 2, \dots, B\}$ ▷ sample actions and evaluate Q
- 12: $\theta \leftarrow \theta + \beta \nabla_\theta \rho^M(q_{1:B})$ ▷ update policy
- 13: $\bar{\phi}_j \leftarrow \tau \phi_j + (1 - \tau) \bar{\phi}_j, j \in \{1, 2\}$ ▷ update target critics
- 14: **end for**
- 15: **end for**

267 **Actor update.** For policy optimization, let $\pi_\theta(a | s) = \mathcal{N}(\mu_\theta(s), \sigma_\theta(s))$, where $\mu_\theta(s)$ and $\sigma_\theta(s)$
 268 are neural-network outputs. We sample $\varepsilon_{1:B} \sim \mathcal{N}(0, 1)$ and construct actions as $a_{1:B} = \mu_\theta(s) +$
 269 $\sigma_\theta(s)\varepsilon_{1:B}$. We then evaluate the Q-values $q_{1:B} = Q_\phi(s, a_{1:B})$. The actor loss is given by

$$L_{\text{actor}}^M(\theta) := -\mathbb{E}_{s \sim \mathcal{D}, \varepsilon_{1:B} \sim \mathcal{N}(0,1)} [\rho^M(q_{1:B})]. \quad (4)$$

270 We optimize the policy parameters via automatic differentiation of $L_{\text{actor}}^M(\theta)$. We refer to this algo-
 271 rithm as **ReMax Actor-Critic (ReMAC)**, and the overall procedure is summarized in Alg. 1.

272 **Remarks on the implementation.** To reduce overestimation, we use clipped double Q-learning
 273 (Haarnoja et al., 2018; Fujimoto et al., 2018) with two critics and a minimum in the target and
 274 the actor loss. In practice, the implementation can be obtained by removing the "soft" components
 275 from Soft Actor-Critic (SAC) (Haarnoja et al., 2018)—namely, entropy bonuses in the critic targets
 276 and temperature tuning—and replacing the actor loss with our ReMax loss. Note that this is a
 277 minimal instantiation of ReMax as an off-policy actor-critic algorithm, and many extensions are
 278 possible. Among them, a promising direction is to maintain a posterior over the Q-function (Osband
 279 et al., 2016; 2018; 2019), making return uncertainty explicit as in ReMax (Koyamada et al., 2023;
 280 Nishimori et al., 2026). This may yield *deeper*, more structured exploration, which is crucial for
 281 sparse-reward settings.

282 7 Experiments

283 The primary goal of our empirical evaluation is *not* to establish ReMAC as a new state-of-the-art
 284 exploration method for continuous control. Instead, we conduct proof-of-concept experiments to
 285 answer the following questions.

- 286 1. Does ReMAC achieve performance comparable to SAC in continuous-control tasks?
- 287 2. Do the entropy-related behaviors predicted by our analysis, higher policy entropy with larger M
 288 and mitigation of the damping effect by larger ε in Adam, appear in practical deep RL training?

289 **Setup.** We consider six continuous-control tasks from Brax (Freeman et al., 2021): Ant, HalfChee-
 290 tah, Hopper, Reacher, Swimmer, and Walker2d. Each agent is trained for 3 million environment
 291 steps. We report the average episodic return over 128 evaluation episodes and the standard error
 292 across 10 random seeds.

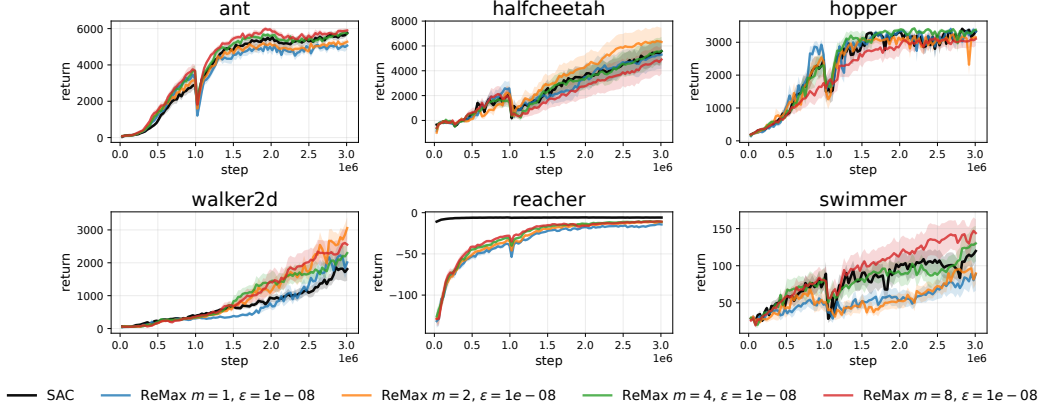


Figure 3: Average return of SAC and ReMAC with different M under Adam’s default $\varepsilon = 1e - 8$. The shaded area shows the standard error across 10 random seeds. ReMAC with $M > 1$ achieves performance comparable to SAC.

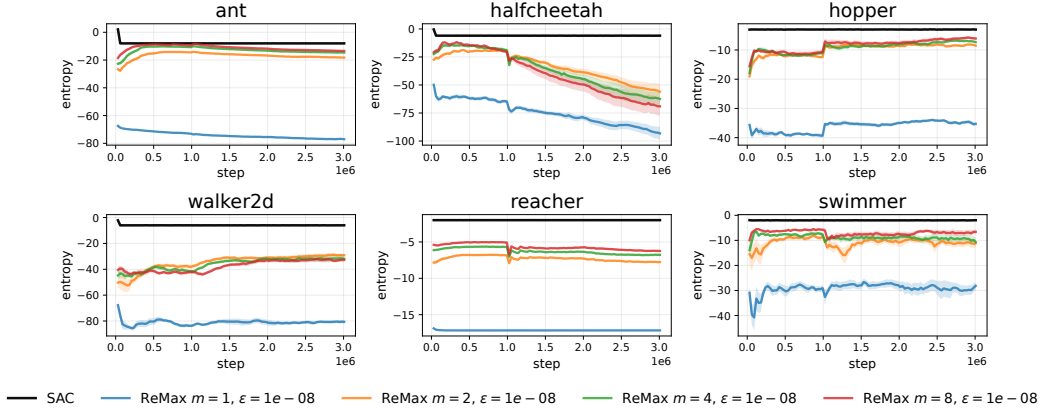


Figure 4: Average policy entropy of SAC and ReMAC with different M under Adam’s default $\varepsilon = 1e - 8$. The shaded area shows the standard error across 10 random seeds. $M > 1$ generally yields higher entropy than $M = 1$, and entropy tends to increase with M (most clearly in Ant).

293 **Baselines.** We use SAC (Haarnoja et al., 2018) as the baseline to verify that ReMAC maintains
 294 competitive performance. Since SAC is a strong and widely used continuous-control method, com-
 295 parable performance suffices for our proof-of-concept goal. Our implementation builds on `rejax`
 296 (Liesen et al., 2024), which provides a vectorized SAC training pipeline and tuned configurations
 297 for each Brax task; we adopt these configurations as our baseline. We evaluate ReMAC with retry
 298 budgets $M \in \{1, 2, 4, 8\}$. For $M = 1, 2, 4$ we set the action-sample batch size $B = 8$, and for
 299 $M = 8$ we set $B = 16$. For fair comparison with SAC, we use Adam’s default $\varepsilon = 1e - 8$ in the
 300 main results. We sweep the learning rate in $\{1e - 4, 3e - 4, 5e - 4, 1e - 3\}$ with 3 random seeds
 301 and select a value that performs reasonably well across environments and M for fixed $\varepsilon = 1e - 8$.
 302 To study the role of Adam’s ε (Sec. 5.3), we additionally vary $\varepsilon \in \{1e - 8, 1e - 2, 1e - 1, 1.0\}$ for
 303 all M . Additional details are provided in App. C.1.

304 **Main results.** Fig. 3 reports the average return with Adam’s default $\varepsilon = 1e - 8$. Across tasks,
 305 ReMAC with $M > 1$ achieves performance comparable to SAC. In several environments (e.g., Ant,
 306 Reacher, Swimmer, and Walker2d), larger retry budgets ($M = 4, 8$) achieve higher average return
 307 than $M = 1$. Fig. 4 reports the average policy entropy for different M . $M > 1$ yields higher
 308 entropy than $M = 1$, and entropy generally increases with M (most clearly in Ant and Swimmer).

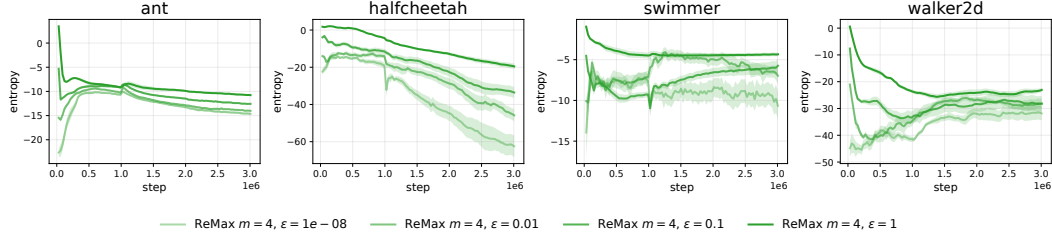


Figure 5: Average policy entropy of ReMAC with $M = 4$ and different $\varepsilon \in \{1e-8, 1e-2, 1e-1, 1.0\}$ in Adam for HalfCheetah, Ant, Swimmer, and Walker2d. Larger ε consistently yields higher entropy, consistent with Sec. 5.3.

This matches the ReMax property identified in Sec. 5.1, where larger M maintains higher entropy. In some environments such as HalfCheetah, however, larger M leads to lower entropy late in training (e.g., $M = 8$ becomes lower than $M = 4$ and $M = 2$). We hypothesize that this results from Adam’s normalization counteracting the damping effect of ReMax (Sec. 5.3). Finally, although ReMAC with $M > 1$ increases entropy relative to $M = 1$, its entropy remains lower than SAC, which is expected because SAC’s critic incorporates future entropy (Haarnoja et al., 2018).

The effect of ε in Adam. To verify whether the damping effect can be recovered by larger ε , we plot the entropy for $M = 4$ with $\varepsilon \in \{1e-8, 1e-2, 1e-1, 1.0\}$ in Fig. 5. Larger ε consistently produces higher entropy during training. This effect is most pronounced in HalfCheetah, where entropy rapidly decreases with $\varepsilon = 1e-8$ but remains higher with $\varepsilon = 1.0$. These results support our hypothesis that Adam’s ε mitigates the practical impact of gradient damping by weakening normalization. Additional results are provided in App. C.2.

The effect of learning rate. Because larger ε also changes the effective step size, higher entropy may not be solely due to reduced damping. To disentangle these effects, we perform a learning-rate sweep for HalfCheetah with $M = 4$ and $\varepsilon = 1.0$, varying the learning rate in $\{1e-4, 3e-4, 5e-4, 1e-3\}$, where the default learning rate for HalfCheetah is $1e-4$. Fig. 6 shows the entropy and return for different learning rates. The red-dotted line indicates the best return and the lowest entropy for $M = 4$, $\varepsilon = 1e-8$, and $\alpha = 1e-4$. Entropy decays faster with larger learning rates, albeit entropy remains substantially higher than the lowest entropy observed with $\varepsilon = 1e-8$ and $\alpha = 1e-4$. We also provide learning-rate sweeps for Swimmer and Walker2d (default learning rate $1e-4$) in App. C.2.

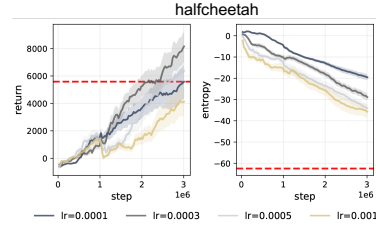


Figure 6: Entropy and return with $M = 4$ and $\varepsilon = 1.0$ under different learning rates for HalfCheetah.

8 Conclusion

In this study, we introduced the ReMax objective to continuous action spaces. We showed that ReMax reshapes the gradient landscape, guiding the policy toward higher entropy. We first demonstrated this effect via a toy vector-field analysis and then provided a theoretical explanation. On the practical side, we instantiated ReMax in an off-policy actor-critic algorithm, ReMax Actor-Critic (ReMAC), which can be implemented with minimal modifications to SAC. Empirically, we provided empirical evidence that ReMAC can control the entropy of the policy without entropy regularization and achieves performance comparable to SAC.

Limitations and future work. Our study has both theoretical and practical limitations. Theoretically, our analysis assumes strong convexity and smoothness of the reward function, which may not

hold in practice. Although the goal of our analysis is to build intuition, extending it to more general settings would help better understand the exploration-encouraging properties of ReMax. Practically, our implementation is minimal and could be improved by incorporating additional exploration mechanisms. Among them, modeling the posterior over the Q-function (Osband et al., 2016; 2018; 2019) fits naturally with ReMax, which explicitly considers return uncertainty. As suggested by Nishimori et al. (2026), this direction may enable deeper, more structured exploration, which is particularly important in sparse-reward settings. Overall, there is ample room for further exploration in this direction. We hope that our analysis of ReMax’s gradient properties and ReMAC’s comparable performance to SAC with minimal modifications will provide a foundation for future work.

References

- Marc Bellemare, Sriram Srinivasan, Georg Ostrovski, Tom Schaul, David Saxton, and Remi Munos. Unifying Count-Based Exploration and Intrinsic Motivation. In *Advances in Neural Information Processing Systems*, 2016.
- Yuri Burda, Harrison Edwards, Amos Storkey, and Oleg Klimov. Exploration by random network distillation. In *International Conference on Learning Representations*, 2019.
- Xinyue Chen, Che Wang, Zijian Zhou, and Keith Ross. Randomized ensembled double q-learning: Learning fast without a model. *arXiv preprint arXiv:2101.05982*, 2021.
- Zhipeng Chen, Xiaobo Qin, Youbin Wu, Yue Ling, Qinghao Ye, Wayne Xin Zhao, and Guang Shi. Pass@ k training for adaptively balancing exploration and exploitation of large reasoning models. *arXiv preprint arXiv:2508.10751*, 2025.
- Petros Christodoulou. Soft actor-critic for discrete action settings. *arXiv preprint arXiv:1910.07207*, 2019.
- Thomas Degris, Martha White, and Richard S Sutton. Off-policy actor-critic. *arXiv preprint arXiv:1205.4839*, 2012.
- Meire Fortunato, Mohammad Gheshlaghi Azar, Bilal Piot, Jacob Menick, Ian Osband, Alex Graves, Vlad Mnih, Remi Munos, Demis Hassabis, Olivier Pietquin, et al. Noisy networks for exploration. *arXiv preprint arXiv:1706.10295*, 2017.
- C Daniel Freeman, Erik Frey, Anton Raichuk, Sertan Girgin, Igor Mordatch, and Olivier Bachem. Brax—a differentiable physics engine for large scale rigid body simulation. *arXiv preprint arXiv:2106.13281*, 2021.
- Scott Fujimoto, Herke Hoof, and David Meger. Addressing function approximation error in actor-critic methods. In *International conference on machine learning*, pp. 1587–1596. PMLR, 2018.
- Evan Greensmith, Peter L Bartlett, and Jonathan Baxter. Variance reduction techniques for gradient estimates in reinforcement learning. *Journal of Machine Learning Research*, 5(Nov):1471–1530, 2004.
- Tuomas Haarnoja, Aurick Zhou, Pieter Abbeel, and Sergey Levine. Soft Actor-Critic: Off-Policy Maximum Entropy Deep Reinforcement Learning with a Stochastic Actor. In *International Conference on Machine Learning*, 2018.
- Jubayer Ibn Hamid, Ifdita Hasan Orney, Ellen Xu, Chelsea Finn, and Dorsa Sadigh. Polychromic objectives for reinforcement learning. *arXiv preprint arXiv:2509.25424*, 2025.
- Takuya Hiraoka, Takahisa Imagawa, Taisei Hashimoto, Takashi Onishi, and Yoshimasa Tsuruoka. Dropout q-functions for doubly efficient reinforcement learning. *arXiv preprint arXiv:2110.02034*, 2021.

- Rein Houthooft, Xi Chen, Xi Chen, Yan Duan, John Schulman, Filip De Turck, and Pieter Abbeel. VIME: Variational Information Maximizing Exploration. In *Advances in Neural Information Processing Systems*, 2016.
- Haque Ishfaq, Yixin Tan, Yu Yang, Qingfeng Lan, Jianfeng Lu, A Rupam Mahmood, Doina Precup, and Pan Xu. More efficient randomized exploration for reinforcement learning via approximate sampling. *arXiv preprint arXiv:2406.12241*, 2024.
- Haque Ishfaq, Guangyuan Wang, Sami Nur Islam, and Doina Precup. Langevin soft actor-critic: Efficient exploration through uncertainty-driven critic learning. *arXiv preprint arXiv:2501.17827*, 2025.
- Martin Jankowiak and Fritz Obermeyer. Pathwise derivatives beyond the reparameterization trick. In *International conference on machine learning*, pp. 2235–2244. PMLR, 2018.
- Diederik P Kingma and Jimmy Ba. Adam: A method for stochastic optimization. *arXiv preprint arXiv:1412.6980*, 2014.
- Diederik P Kingma and Max Welling. Auto-encoding variational bayes. *arXiv preprint arXiv:1312.6114*, 2013.
- Sotetsu Koyamada, Paavo Parmas, Tadashi Kozuno, and Shin Ishii. Emergence of exploration in policy gradient reinforcement learning via resetting, 2023. URL https://openreview.net/forum?id=GKsNIC_mQRG.
- Hojoon Lee, Dongyoon Hwang, Donghu Kim, Hyunseung Kim, Jun Jet Tai, Kaushik Subramanian, Peter R Wurman, Jaegul Choo, Peter Stone, and Takuma Seno. Simba: Simplicity bias for scaling up parameters in deep reinforcement learning. *arXiv preprint arXiv:2410.09754*, 2024.
- Jarek Liesen, Chris Lu, and Robert Lange. rejax, 2024. URL <https://github.com/keraJLi/rejax>.
- Timothy Paul Lillicrap, Jonathan James Hunt, Alexander Pritzel, Nicolas Manfred Otto Heess, Tom Erez, Yuval Tassa, David Silver, and Daniel Pieter Wierstra. Continuous control with deep reinforcement learning, September 15 2020. US Patent 10,776,692.
- Sam Lobel, Akhil Bagaria, and George Konidaris. Flipping coins to estimate pseudocounts for exploration in reinforcement learning. In *International Conference on Machine Learning*, pp. 22594–22613. PMLR, 2023.
- Michal Nauman, Mateusz Ostaszewski, Krzysztof Jankowski, Piotr Miłoś, and Marek Cygan. Bigger, regularized, optimistic: scaling for compute and sample efficient continuous control. *Advances in neural information processing systems*, 37:113038–113071, 2024.
- Soichiro Nishimori, Paavo Parmas, Sotetsu Koyamada, Tadashi Kozuno, Toshinori Kitamura, Shin Ishii, and Yutaka Matsuo. Emergence of exploration in policy gradient reinforcement learning via retrying, 2026. URL <https://openreview.net/forum?id=msX2KKICRr>.
- Ian Osband, Charles Blundell, Alexander Pritzel, and Benjamin Van Roy. Deep exploration via bootstrapped dqn. In *Advances in neural information processing systems*, volume 29, pp. 4026–4034, 2016.
- Ian Osband, John Aslanides, and Albin Cassirer. Randomized prior functions for deep reinforcement learning. *Advances in neural information processing systems*, 31, 2018.
- Ian Osband, Benjamin Van Roy, Daniel J Russo, and Zheng Wen. Deep exploration via randomized value functions. *Journal of Machine Learning Research*, 20(124):1–62, 2019.
- Ian Osband, Zheng Wen, Seyed Mohammad Asghari, Vikranth Dwaracherla, Morteza Ibrahimi, Xiuyuan Lu, and Benjamin Van Roy. Approximate thompson sampling via epistemic neural networks. In *Uncertainty in Artificial Intelligence*, pp. 1586–1595. PMLR, 2023.

- 431 Georg Ostrovski, Marc G Bellemare, Aäron Oord, and Rémi Munos. Count-based exploration with
 432 neural density models. In *International conference on machine learning*, pp. 2721–2730. PMLR,
 433 2017.
- 434 Paavo Parmas. Total stochastic gradient algorithms and applications in reinforcement learning. *Ad-*
 435 *vances in Neural Information Processing Systems*, 31, 2018.
- 436 Paavo Parmas and Masashi Sugiyama. A unified view of likelihood ratio and reparameterization
 437 gradients. In *International Conference on Artificial Intelligence and Statistics*, pp. 4078–4086.
 438 PMLR, 2021.
- 439 Paavo Parmas, Carl Edward Rasmussen, Jan Peters, and Kenji Doya. Pippis: Flexible model-based
 440 policy search robust to the curse of chaos. In *International Conference on Machine Learning*, pp.
 441 4065–4074. PMLR, 2018.
- 442 Deepak Pathak, Pulkrit Agrawal, Alexei A. Efros, and Trevor Darrell. Curiosity-Driven Exploration
 443 by Self-Supervised Prediction. In *International Conference on Machine Learning*, 2017.
- 444 Matthias Plappert, Rein Houthooft, Prafulla Dhariwal, Szymon Sidor, Richard Y Chen, Xi Chen,
 445 Tamim Asfour, Pieter Abbeel, and Marcin Andrychowicz. Parameter space noise for exploration.
 446 *arXiv preprint arXiv:1706.01905*, 2017.
- 447 Danilo Jimenez Rezende, Shakir Mohamed, and Daan Wierstra. Stochastic backpropagation and ap-
 448 proximate inference in deep generative models. In *International conference on machine learning*,
 449 pp. 1278–1286. PMLR, 2014.
- 450 Jürgen Schmidhuber. Formal Theory of Creativity, Fun, and Intrinsic Motivation (1990–2010). *IEEE*
 451 *Transactions on Autonomous Mental Development*, 2(3):230–247, 2010.
- 452 Bhavya Sukhija, Stelian Coros, Andreas Krause, Pieter Abbeel, and Carmelo Sferrazza. Maxinforl:
 453 Boosting exploration in reinforcement learning through information gain maximization. *arXiv*
 454 *preprint arXiv:2412.12098*, 2024.
- 455 Richard S. Sutton and Andrew G. Barto. *Reinforcement Learning: An Introduction*. The MIT Press,
 456 2nd edition, 2018.
- 457 Haoran Tang, Rein Houthooft, Davis Foote, Adam Stooke, OpenAI Xi Chen, Yan Duan, John Schul-
 458 man, Filip DeTurck, and Pieter Abbeel. #Exploration: A Study of Count-Based Exploration for
 459 Deep Reinforcement Learning. In *Advances in Neural Information Processing Systems*, 2017.
- 460 Yunhao Tang, Kunhao Zheng, Gabriel Synnaeve, and Rémi Munos. Optimizing language models for
 461 inference time objectives using reinforcement learning. *arXiv preprint arXiv:2503.19595*, 2025.
- 462 Christian Walder and Deep Karkhanis. Pass@ k policy optimization: Solving harder reinforcement
 463 learning problems. *arXiv preprint arXiv:2505.15201*, 2025.
- 464 Ronald J Williams. Simple statistical gradient-following algorithms for connectionist reinforcement
 465 learning. *Machine Learning*, 8(3):229–256, May 1992.
- 466 Shangdong Zhang and Hengshuai Yao. Ace: An actor ensemble algorithm for continuous control
 467 with tree search. In *Proceedings of the AAAI Conference on Artificial Intelligence*, volume 33,
 468 pp. 5789–5796, 2019.

Supplementary Materials

The following content was not necessarily subject to peer review.

A Proofs of Propositions in Sec. 5

In this section, we provide additional results and proofs of the propositions in Sec. 5. Before presenting the proofs, we summarize useful lemmas derived from the assumptions. Though some of the lemmas are straightforward, we include proofs for completeness.

Lemma 1 (Upper and lower bounds on c). Assume Assumptions 1, 2, and 3 hold. For all $a \in \mathbb{R}$, we have:

$$\frac{\lambda}{2}a^2 \leq c(a) \leq \frac{L}{2}a^2, \quad (5)$$

Proof. By λ -strong convexity, for any $a \in \mathbb{R}$, $c(a) \geq c(0) + c'(0)a + \frac{\lambda}{2}a^2 = \frac{\lambda}{2}a^2$. By L -smoothness, for any $a \in \mathbb{R}$, $c(a) \leq c(0) + c'(0)a + \frac{L}{2}a^2 = \frac{L}{2}a^2$. $\square$

Lemma 2 (Bounds on c'). Assume Assumptions 1, 2, and 3 hold. Then, for any $a \geq 0$,

$$\lambda a \leq c'(a) \leq La.$$

Proof. Since $c'(0) = 0$ and $\lambda \leq c''(t) \leq L$ for all $t \in \mathbb{R}$, for any $a \geq 0$ we have $c'(a) - c'(0) = \int_0^a c''(t) dt$. Hence, $\int_0^a \lambda dt \leq c'(a) \leq \int_0^a L dt$, which gives $\lambda a \leq c'(a) \leq La$. $\square$

Lemma 3 (Upper bound on c'). Assume Assumptions 1 and 3 hold. For all $a \in \mathbb{R}$, we have:

$$|c'(a)| \leq L|a|. \quad (6)$$

Proof. By L -smoothness, for any $a \in \mathbb{R}$, $|c'(a)| = |c'(a) - c'(0)| \leq L|a - 0| = L|a|$. $\square$

Lemma 4 (Well-definedness and differentiation of J^M). Assume Assumptions 1, 2, and 3 hold, and let $\sigma > 0$. Then $J^M(\mu, \sigma)$ is finite and differentiable in (μ, σ) . Moreover,

$$\partial_\mu J^M(\mu, \sigma) = \mathbb{E}[-c'(A_{m^*})], \quad \partial_\sigma J^M(\mu, \sigma) = \mathbb{E}[-c'(A_{m^*})\varepsilon_{m^*}],$$

where $m^* \in \arg \min_{m=1:M} c(A_m)$.

Proof. By Lemma 1, for all $a \in \mathbb{R}$,

$$0 \leq c(a) \leq \frac{L}{2}a^2.$$

Hence,

$$-\frac{L}{2} \max_{m=1:M} A_m^2 \leq \max_{m=1:M} (-c(A_m)) \leq 0.$$

Since $M < \infty$ and each A_m is Gaussian, $\mathbb{E}[\max_{m=1:M} A_m^2] < \infty$, so $J^M(\mu, \sigma)$ is well-defined.

For a fixed realization of $\varepsilon_{1:M}$, define

$$F(\mu, \sigma) := \max_{m=1:M} (-c(\mu + \sigma\varepsilon_m)).$$

This is the maximum of finitely many C^1 functions. Moreover, ties occur with probability zero, since the A_m are continuous random variables and c is strictly convex. Therefore, for almost every realization, the maximizer is unique, and

$$\partial_\mu F(\mu, \sigma) = -c'(A_{m^*}), \quad \partial_\sigma F(\mu, \sigma) = -c'(A_{m^*})\varepsilon_{m^*}.$$

By Lemma 3,

$$|c'(A_{m^*})| \leq L|A_{m^*}| \leq L\left(|\mu| + \sigma \max_{m=1:M} |\varepsilon_m|\right),$$

and

$$|c'(A_{m^*})\varepsilon_{m^*}| \leq L|A_{m^*}| |\varepsilon_{m^*}| \leq L\left(|\mu| + \sigma \max_{m=1:M} |\varepsilon_m|\right) \max_{m=1:M} |\varepsilon_m|.$$

Both dominating random variables are integrable because $M < \infty$ and Gaussian moments are finite. Hence differentiation can be interchanged with expectation by dominated convergence, yielding the stated formulas. $\square$

A.1 Proof of Proposition 2

Proof. By Lemma 4,

$$\partial_\sigma J^1(\mu, \sigma) = \mathbb{E}[-c'(\mu + \sigma\varepsilon)].$$

Let $f(\varepsilon) = c'(\mu + \sigma\varepsilon)$. Then f is differentiable and, by Assumption 3, $|f'(\varepsilon)| = \sigma|c''(\mu + \sigma\varepsilon)| \leq \sigma L$. Hence Stein's lemma (Gaussian integration by parts) applies, yielding

$$\mathbb{E}[c'(\mu + \sigma\varepsilon)\varepsilon] = \mathbb{E}[f'(\varepsilon)] = \sigma\mathbb{E}[c''(\mu + \sigma\varepsilon)].$$

Therefore, $\partial_\sigma J^1(\mu, \sigma) = -\sigma\mathbb{E}[c''(\mu + \sigma\varepsilon)]$. Since c is λ -strongly convex, we have $c''(a) \geq \lambda$ for all $a \in \mathbb{R}$. Taking expectations gives $\partial_\sigma J^1(\mu, \sigma) \leq -\lambda\sigma < 0$.

This proves that for $M = 1$, the objective strictly penalizes variance and drives σ toward zero. $\square$

A.2 Proof of Proposition 1

Proof. By Lemma 4., the gradient of the objective with respect to σ is given by $\partial_\sigma J^M(\mu, \sigma) = \mathbb{E}[-c'(A_{m^*})\varepsilon_{m^*}]$. We split this expectation into the event $E = \{\mu + \sigma\varepsilon_{(1)} \geq 0\}$ and its complement E^c .

First, conditionally on the event E , we have $A_m = \mu + \sigma\varepsilon_m \geq 0$ for all m . By Assumptions 1 and 2, $c'(0) = 0$ and $c''(a) \geq \lambda > 0$, meaning c is strictly monotonically increasing on $\mathbb{R}_{\geq 0}$. Therefore, the action that minimizes $c(A_m)$ is simply the one with the smallest A_m . Since μ and $\sigma > 0$ are constants across m , this corresponds to the action with the minimum noise, yielding $m^* = \arg \min_{m=1:M} \varepsilon_m$ and $A_{m^*} = \mu + \sigma\varepsilon_{(1)}$.

Next, we bound the term $-c'(A_{m^*})\varepsilon_{(1)}$. Using Lemma 2, the derivative for any $a \geq 0$ is bounded by $\lambda a \leq c'(a) \leq La$. Let $\varepsilon_{(1)}^- = \max(0, -\varepsilon_{(1)})$ and $\varepsilon_{(1)}^+ = \max(0, \varepsilon_{(1)})$. We consider two cases depending on the sign of $\varepsilon_{(1)}$:

- If $\varepsilon_{(1)} < 0$: We multiply the lower bound $c'(A_{m^*}) \geq \lambda A_{m^*}$ by the positive scalar $(-\varepsilon_{(1)})$. Using $\varepsilon_{(1)}^- = -\varepsilon_{(1)}$, we obtain:

$$-c'(A_{m^*})\varepsilon_{(1)} \geq \lambda(\mu + \sigma\varepsilon_{(1)})(-\varepsilon_{(1)}) = \lambda\mu\varepsilon_{(1)}^- - \lambda\sigma(\varepsilon_{(1)}^-)^2.$$

- If $\varepsilon_{(1)} \geq 0$: We multiply the upper bound $c'(A_{m^*}) \leq LA_{m^*}$ by the negative scalar $(-\varepsilon_{(1)})$, which flips the inequality to $-c'(A_{m^*}) \geq -LA_{m^*}$. Using $\varepsilon_{(1)}^+ = \varepsilon_{(1)}$, we obtain:

$$-c'(A_{m^*})\varepsilon_{(1)} \geq -L(\mu + \sigma\varepsilon_{(1)})\varepsilon_{(1)} = -L\mu\varepsilon_{(1)}^+ - L\sigma(\varepsilon_{(1)}^+)^2.$$

Combining these two cases and using $\max(\lambda, L)$ for the quadratic penalty term, we obtain the unified lower bound on E :

$$-c'(A_{m^*})\varepsilon_{m^*} \mathbb{I}_E \geq \left(\mu(\lambda\varepsilon_{(1)}^- - L\varepsilon_{(1)}^+) - \sigma \max(\lambda, L)\varepsilon_{(1)}^2\right) \mathbb{I}_E.$$

Taking the expectation of this bound and adding the residual expectation on E^c gives the stated inequality in the proposition. $\square$

521 A.3 A unified version of Proposition 1

522 We stated Proposition 1 using the event $E = \{\mu + \sigma\varepsilon_{(1)} \geq 0\}$, which is convenient when the mean
 523 is far on the positive side (e.g., $\mu \gg 0$). An analogous statement holds when the mean is far on the
 524 negative side (e.g., $\mu \ll 0$). Below we provide a unified formulation that covers both cases with
 525 minimal additional notation.

526 *Remark 1* (Selection rule on one side of the optimum). Under Assumptions 1 and 2, $a = 0$ is
 527 the unique minimizer of c . Hence c is strictly increasing on $\mathbb{R}_{\geq 0}$ and strictly decreasing on $\mathbb{R}_{\leq 0}$.
 528 Therefore, if all sampled actions lie on the same side of 0, the minimizer of $c(A_m)$ is the action
 529 closest to 0: if $A_m \geq 0$ for all m , then $m^* = \arg \min_m A_m$; if $A_m \leq 0$ for all m , then $m^* =$
 530 $\arg \max_m A_m$.

531 **Proposition 4** (Entropy Increase (unified)). Assume Assumptions 1, 2, and 3 hold. Let $s =$
 532 $\text{sign}(\mu) \in \{-1, 1\}$ (and define $s = 1$ if $\mu = 0$). Define the transformed noises $\tilde{\varepsilon}_m = s\varepsilon_m$,
 533 and let $\tilde{\varepsilon}_{(1)} = \min_{m=1:M} \tilde{\varepsilon}_m$. Define the event

$$\tilde{E} := \{|\mu| + \sigma\tilde{\varepsilon}_{(1)} \geq 0\},$$

534 which is equivalent to $\{sA_m \geq 0 \text{ for all } m\}$. Then, the gradient of the ReMax objective with
 535 respect to σ satisfies

$$\partial_\sigma J^M(\mu, \sigma) \geq |\mu| \mathbb{E} \left[(\lambda \tilde{\varepsilon}_{(1)}^- - L \tilde{\varepsilon}_{(1)}^+) \mathbb{I}_{\tilde{E}} \right] - \sigma \max(\lambda, L) \mathbb{E} \left[\tilde{\varepsilon}_{(1)}^2 \mathbb{I}_{\tilde{E}} \right] + \mathbb{E} \left[-c'(A_{m^*}) \varepsilon_{m^*} \mathbb{I}_{\tilde{E}^c} \right],$$

536 where $\tilde{\varepsilon}_{(1)}^- = \max(0, -\tilde{\varepsilon}_{(1)})$ and $\tilde{\varepsilon}_{(1)}^+ = \max(0, \tilde{\varepsilon}_{(1)})$.

537 **Proof.** The proof follows the same steps as Proposition 1 after a sign-reparameterization. By
 538 Lemma 4, $\partial_\sigma J^M(\mu, \sigma) = \mathbb{E}[-c'(A_{m^*})\varepsilon_{m^*}]$. Split the expectation into $\tilde{E}$ and $\tilde{E}^c$. On $\tilde{E}$, we
 539 have $sA_m = |\mu| + \sigma\tilde{\varepsilon}_m \geq 0$ for all m . By Remark 1, the selected index minimizes sA_m , hence
 540 $m^* = \arg \min_m \tilde{\varepsilon}_m$ and $sA_{m^*} = |\mu| + \sigma\tilde{\varepsilon}_{(1)}$. The rest of the argument is identical to the original
 541 proof, replacing $(\mu, \varepsilon_{(1)}, E)$ by $(|\mu|, \tilde{\varepsilon}_{(1)}, \tilde{E})$, yielding the stated bound after taking expectations
 542 and adding the residual term on $\tilde{E}^c$. $\square$

543 A.4 Proof of Proposition 3

544 **Proof.** By Lemma 4, the gradient with respect to μ is $\partial_\mu J^M(\mu, \sigma) = \mathbb{E}[-c'(A_{m^*})]$, and the gradient
 545 with respect to σ is $\partial_\sigma J^M(\mu, \sigma) = \mathbb{E}[-c'(A_{m^*})\varepsilon_{m^*}]$. By Lemma 3, we have $|c'(a)| \leq L|a|$, which
 546 implies:

$$\begin{aligned} |\partial_\mu J^M(\mu, \sigma)| &\leq \mathbb{E}[|c'(A_{m^*})|] \leq L \mathbb{E}[|A_{m^*}|], \\ |\partial_\sigma J^M(\mu, \sigma)| &\leq \mathbb{E}[|c'(A_{m^*})||\varepsilon_{m^*}|] \leq L \mathbb{E}[|A_{m^*}||\varepsilon_{m^*}|]. \end{aligned}$$

By definition, m^* minimizes $c(A_m)$, so $c(A_{m^*}) \leq c(A_m)$ for all $m \leq M$. Using Lemma 1, we
 obtain:

$$\frac{\lambda}{2} A_{m^*}^2 \leq c(A_{m^*}) \leq \min_{m=1:M} c(A_m) \leq \min_{m=1:M} \frac{L}{2} A_m^2 = \frac{L}{2} \min_{m=1:M} A_m^2.$$

547 Taking the square root of both sides gives $|A_{m^*}| \leq \sqrt{\frac{L}{\lambda}} \min_{m=1:M} |A_m|$. Substituting this in-
 548 equality back into the expected gradient bounds completes the proof. This demonstrates that the
 549 gradients are constrained by the minimum distance to the optimum, scaled by the condition number
 550 $\sqrt{L/\lambda}$. $\square$

551 B Details of the gradient analysis

552 We provide additional details for the vector field analysis in Sec. 5.1 and the effect of Adam's
 553 normalization in Sec. 5.3.

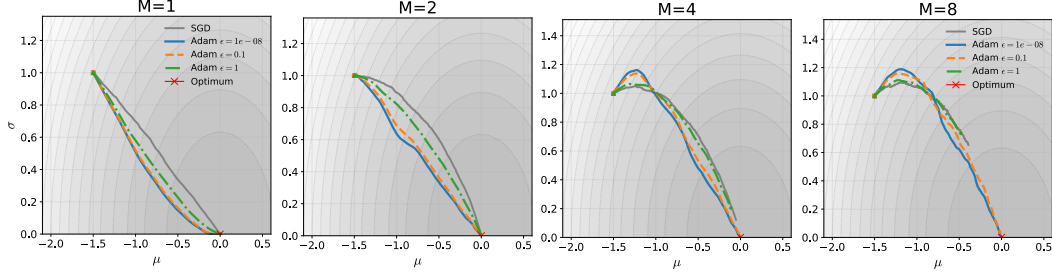


Figure 7: The 2D plot of the parameter (μ, σ) optimized by Adam starting at $(\mu, \sigma) = (-1.5, 1.0)$ with different epsilon values ($\varepsilon \in \{1e-8, 1e-1, 1.0\}$) and SGD. As the vector field indicates, increasing M makes the path deviate more from the optimum and slows convergence. However, with smaller epsilon the convergence near the optimum remains fast, whereas larger epsilon aligns the trajectory with the SGD path that follows the vector field.

554 B.1 Details for the vector field analysis

555 In Sec. 5.1, we visualized the gradient landscape of the ReMax objective to build intuition for its
 556 exploration-encouraging properties. Here we describe the setup and methodology used to generate
 557 these vector fields.

558 We consider a stateless continuous-control problem where the policy is a one-dimensional Gaussian
 559 distribution $\mathcal{N}(\mu, \sigma^2)$. To enable differentiation through sampling, actions are parameterized using
 560 the reparameterization trick, $a_i = \mu + \sigma \varepsilon_i$, where $\varepsilon_i \sim \mathcal{N}(0, 1)$ are i.i.d. standard normal noises.
 561 We use the quadratic deterministic reward $r(a) = -a^2$, making $a = 0$ the unique optimal action.
 562 For entropy regularization, we set $\alpha = 0.5$ and use $r(a) = -a^2 + \alpha \frac{1}{2} \log(2\pi\sigma^2)$.

563 To construct the vector field, we evaluate gradients of the ReMax objective with respect to μ and σ
 564 on a grid defined by $\mu \in [-2, 2]$ and $\sigma \in [0.1, 2]$. For each coordinate (μ, σ) , we sample a batch of
 565 $B = 16$ actions and compute gradients using the reparameterization (RP) estimator with automatic
 566 differentiation. A single stochastic gradient evaluation would produce a noisy vector field. To
 567 approximate the expected gradient landscape $\mathbb{E}[\nabla_{\mu, \sigma} \mathcal{J}^M]$, we therefore average RP gradients over
 568 100 independent Monte Carlo trials for each grid coordinate.

569 B.2 Details for the effect of normalization by Adam

570 In Sec. 5.3, we showed that Adam’s normalization can mitigate the damping effect of ReMax. To
 571 illustrate this, we plot the convergence path of (μ, σ) in the two-dimensional parameter space. As M
 572 increases, the trajectory becomes more gradual around the optimum, consistent with the vector-field
 573 analysis.

574 Furthermore, the consistency with SGD observed for larger ε in Fig. 2 also appears in these trajec-
 575 tories, supporting the argument that increasing ε makes the optimization path follow the vector field
 576 more closely.

577 C Details of the experiments

578 Here we provide additional details for the experiments in Sec. 7.

579 C.1 Setup

580 **Implementation.** The implementation of the ReMAC algorithm is based on the `rejax`¹ imple-
 581 mentation of SAC. We submit the code as the supplementary material.

¹<https://github.com/kerajli/rejax>

Table 1: Selected hyperparameters for ReMAC.

Environment	Learning Rate
HalfCheetah	1e-4
Ant	2e-4
Hopper	3e-4
Reacher	5e-4
Swimmer	1e-4
Walker2d	1e-4

Hyperparameters. We follow the SAC implementation and hyperparameters provided in the rejax library (Liesen et al., 2024), which contains environment-specific tuned configurations for Brax tasks. Listing all hyperparameters would largely duplicate that repository, so we refer readers to the code provided in the environment-wise configuration files in the supplementary material at `brax/configs/` for the exact configuration used in our experiments. The hyperparameters (learning rate, batch size, discount factor, Polyak factor, etc.) were tuned for each environment by the authors of rejax. We tuned the learning rate for ReMAC by sweeping $\{1e-4, 3e-4, 5e-4, 1e-3\}$ with 3 random seeds and selecting a value that performs reasonably well across environments and M for fixed $\varepsilon = 1e-8$. The selected learning rates are provided in Tab. 1. The other hyperparameters are the same as those used for SAC in rejax.

C.2 Additional results

Here we provide additional results for the experiments in Sec. 7.

Entropy and return with different ε We plot the average policy entropy and return with different $\varepsilon \in \{1e-8, 1e-2, 1e-1, 1.0\}$ for $M = 1, 2, 4, 8$ in Fig. 8 to Fig. 15. For all M , entropy increases as ε increases, consistent with Sec. 5.3. The return tends to decrease for larger ε , as expected, because a larger ε reduces the effective step size and can slow learning.

Learning rate sweep. Fig. 16 plots the entropy and return with $M = 4$ and $\varepsilon = 1.0$ under different learning rates $\alpha \in \{1e-4, 3e-4, 5e-4, 1e-3\}$ for Swimmer and Walker2d. As in HalfCheetah (Sec. 7), the default learning rate is $1e-4$. The red-dotted line indicates the best return and lowest entropy for $M = 4$, $\varepsilon = 1e-8$, and $\alpha = 1e-4$. Similar to HalfCheetah, entropy decays faster with larger learning rates. Nevertheless, the entropy remains much higher than the lowest entropy with $\varepsilon = 1e-8$.

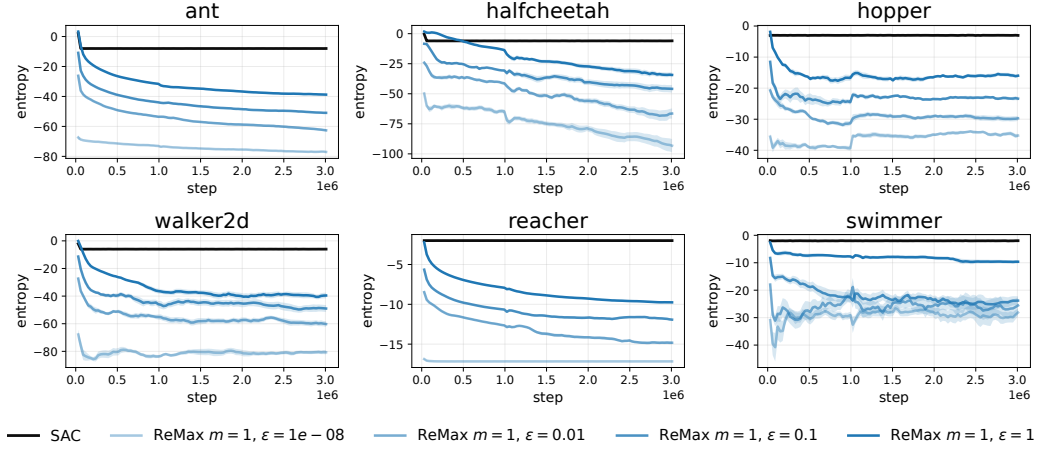


Figure 8: The average policy entropy of ReMAC with $M = 1$ and different $\epsilon \in \{1e-8, 1e-2, 1e-1, 1.0\}$ in Adam for all tasks.

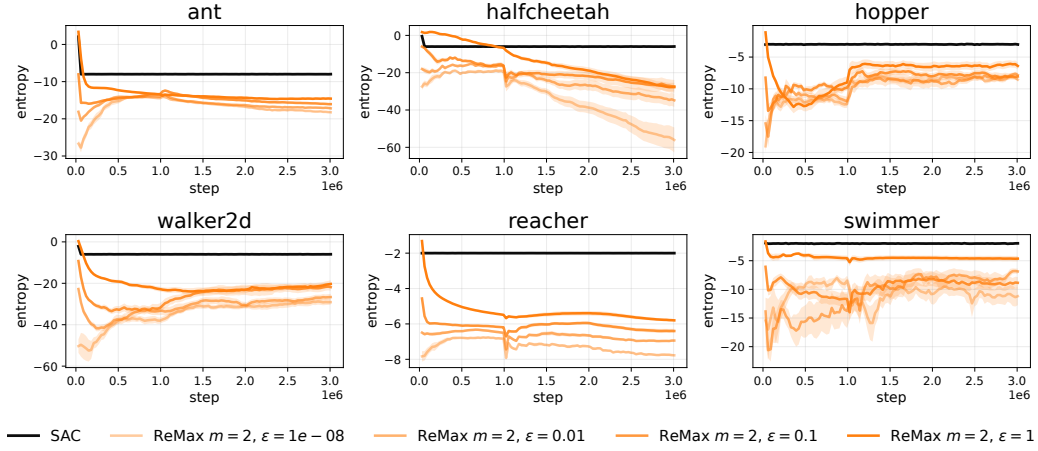


Figure 9: The average policy entropy of ReMAC with $M = 2$ and different $\epsilon \in \{1e-8, 1e-2, 1e-1, 1.0\}$ in Adam for all tasks.

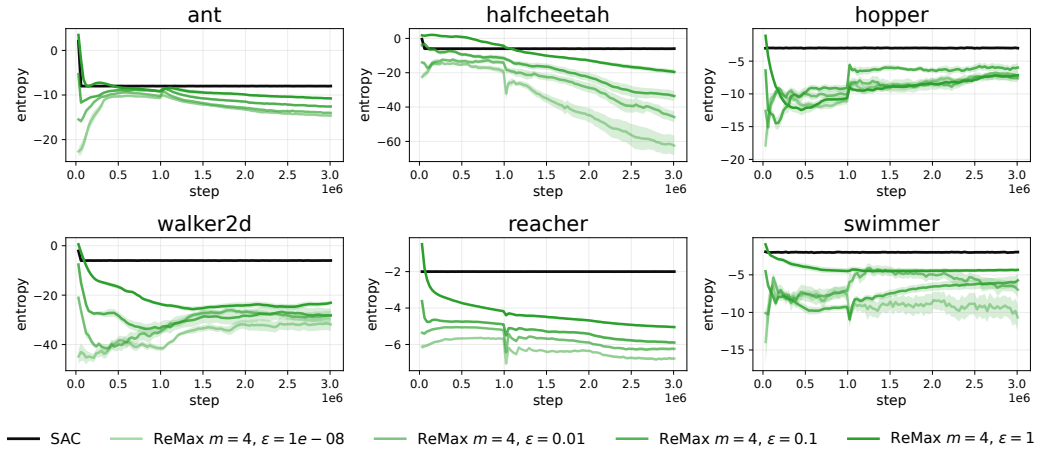


Figure 10: The average policy entropy of ReMAC with $M = 4$ and different $\epsilon \in \{1e-8, 1e-2, 1e-1, 1.0\}$ in Adam for all tasks.

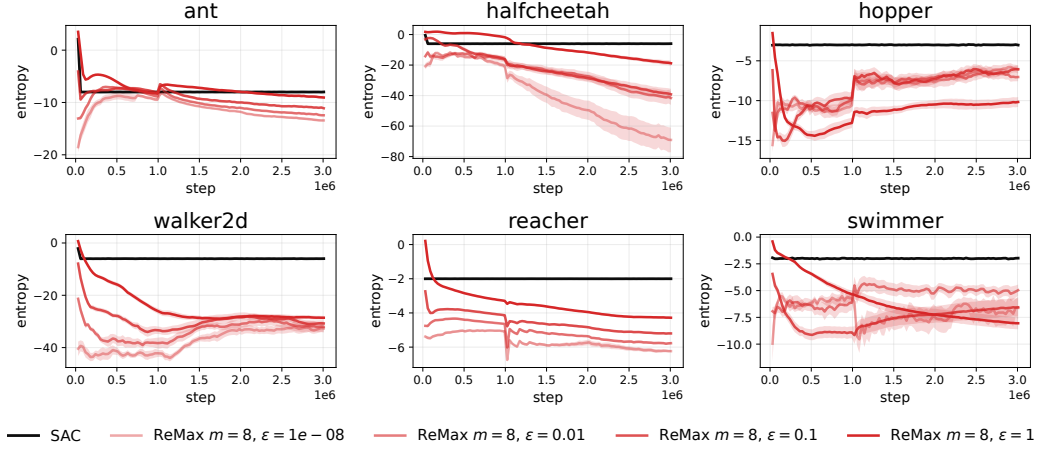


Figure 11: The average policy entropy of ReMAC with $M = 8$ and different $\epsilon \in \{1e-8, 1e-2, 1e-1, 1.0\}$ in Adam for all tasks.

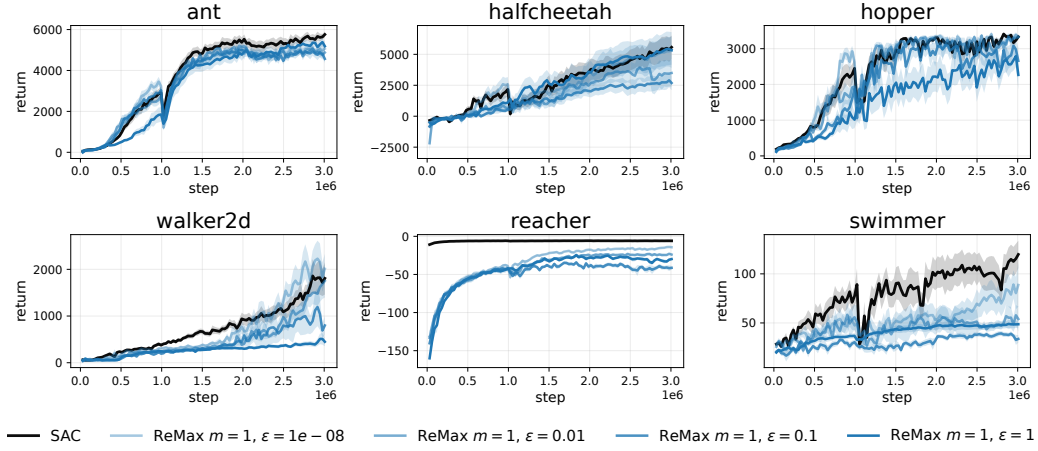


Figure 12: The average return of ReMAC with $M = 1$ and different $\epsilon \in \{1e-8, 1e-2, 1e-1, 1.0\}$ in Adam for all tasks.

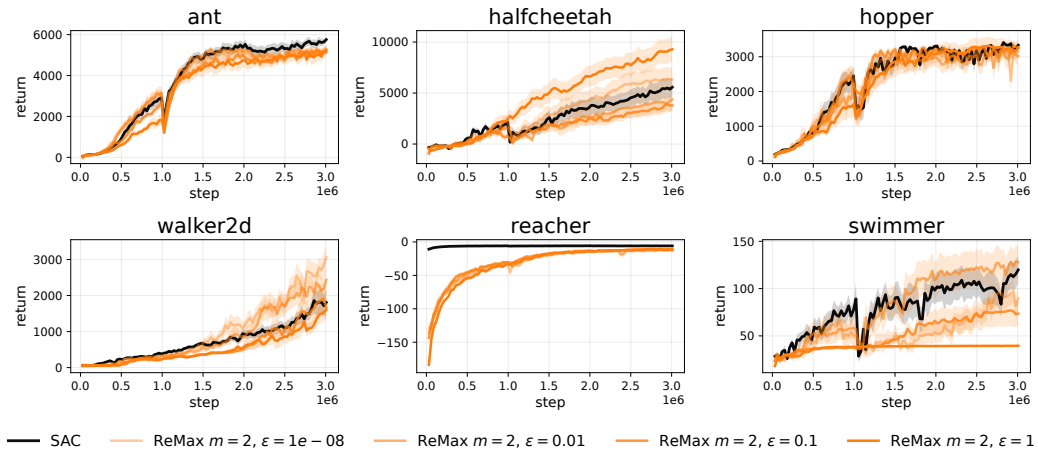


Figure 13: The average return of ReMAC with $M = 2$ and different $\epsilon \in \{1e-8, 1e-2, 1e-1, 1.0\}$ in Adam for all tasks.

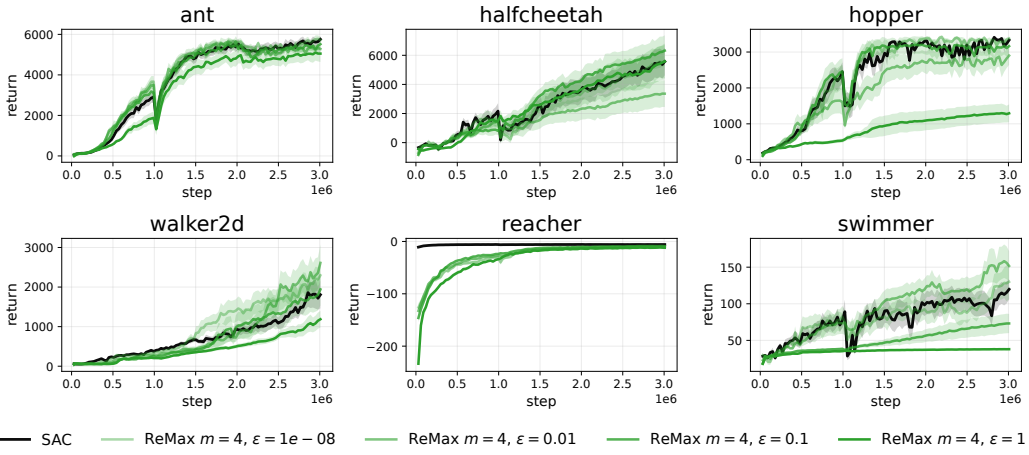


Figure 14: The average return of ReMAC with $M = 4$ and different $\varepsilon \in \{1e-8, 1e-2, 1e-1, 1.0\}$ in Adam for all tasks.

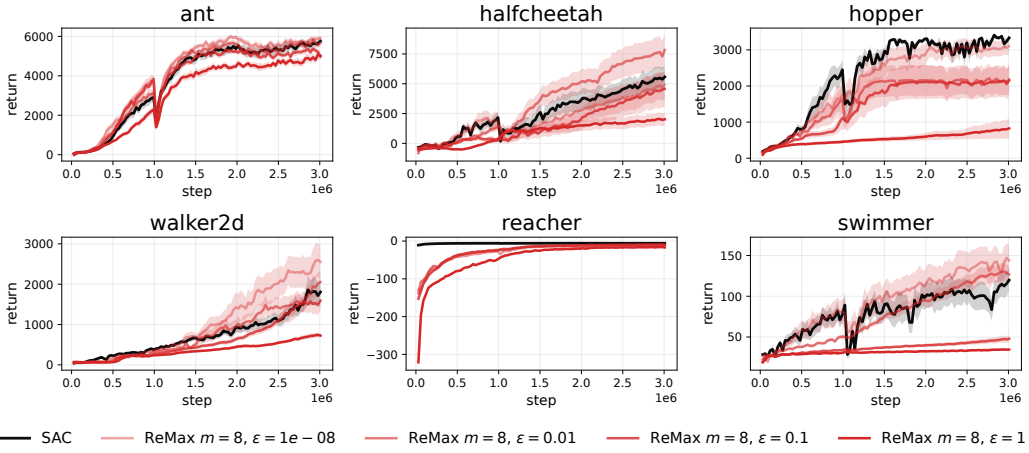


Figure 15: The average return of ReMAC with $M = 8$ and different $\varepsilon \in \{1e-8, 1e-2, 1e-1, 1.0\}$ in Adam for all tasks.

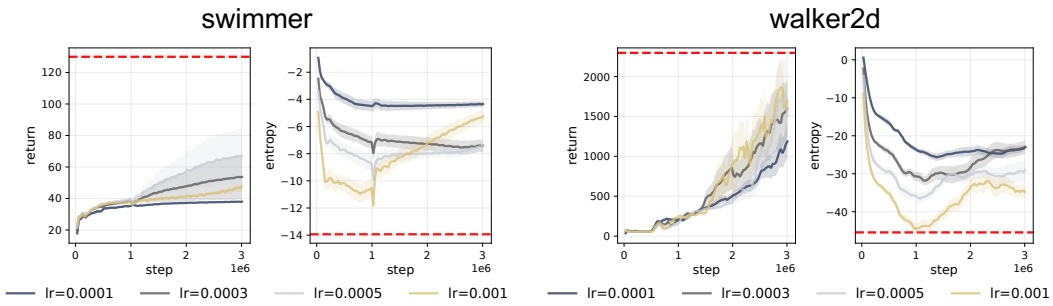


Figure 16: The entropy and return with $M = 4$ and $\varepsilon = 1.0$ under different learning rates in Swimmer.